DB 18.11.25 Cheat Sheet

- **Functional Dependency**
- **Superkey**

MatrN → Name	
A → R	key
A + smth → R	superkey

- **Properties of FD**
- **Splitting Algorithm** $ABC \rightarrow \cancel{C}DE$
- **Closure Algorithm** $A^+ = \{A, \dots\}$
- **Membership Problem** $\{A\}^+ \ni B \overset{if}{\Rightarrow} A \rightarrow B$
- **Key Heuristic**
  1. start with (closure) $\{A\}^+$, A not part of any right side
  2. if $\{A\}^+ = \underline{R} \Rightarrow$ A only key
  3. A is part of any key, try all combinations
- **Cover** ① Splitting ② Remove Augmentations

  $ABC \rightarrow D \rightsquigarrow AB \rightarrow C$
  if $\{AB\}^+ \ni D$

  ③ Remove unnecessary FDs if $AB \rightarrow C$

  still valid?

  and $\{AB\}^+_{FD \setminus AB \rightarrow C} \ni C$
  then $\cancel{AB \rightarrow C}$

Normalization

1NF	no comma separated lists
2NF	AB as key and $A \rightarrow C$ then not 2NF must depend on whole key
3NF	left side superkey or right side prime
BCNF	left side superkey

DB contd. 18.11.25

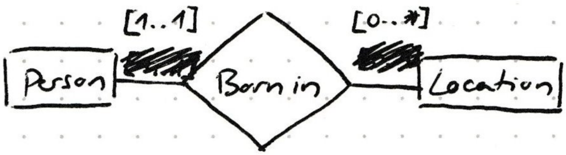
- **Decomposition**
  1. FD that violates BCNF/3NF
  2. $AB \rightarrow C$ — section it off into its own relation
     $R_1 = \{AB\}^+$ — key: AB
     $R_2 = \text{Rest} + AB$
  3. check dependency preservation
  4. check BCNF/3NF

- **Synthesis**
  1. $AB \rightarrow CD \rightsquigarrow R_1 = \{ABCD\}$ — each FD gets its own relation R_n
  2. is $R_1 \subset R_2 \Rightarrow$ Remove R_1
  3. check if key of R is part of R_n
  4. if no $\rightarrow$ add R_{key}

= Cheat Sheet End =

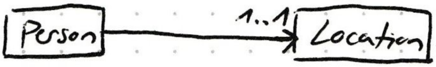
Chapter 6 | Conceptual Database Design

ER-Diagram Notation: Chen, Min-Max / ISO
Crow's Foot



"Every person is born in exactly one location and in every location, zero or any number of persons are born."

cf. UML

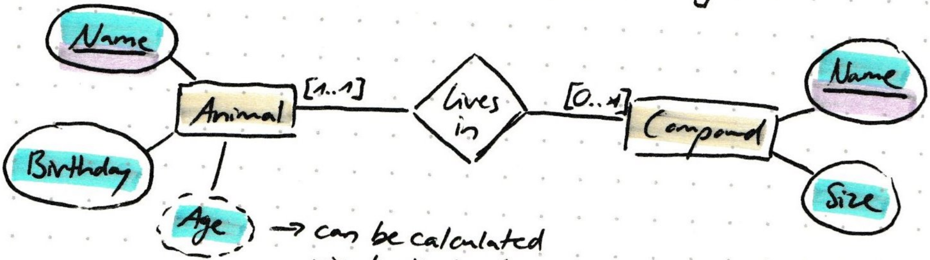


DB contd. 18.11.15

Simple Relationship: one-to-many

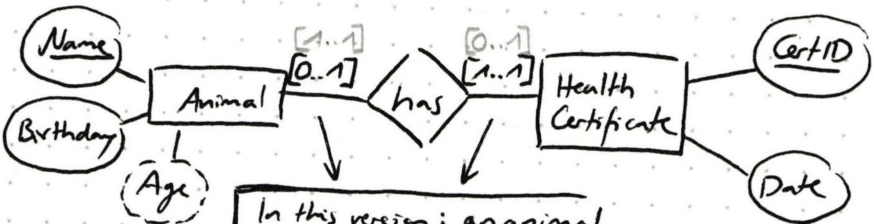
Zoo Example

- entity represents real world elem; becomes relation
  - ↳ represented by its attributes
  - ↳ relationship is represented by key/foreign key
- primary keys are ~~not~~ marked by underlining



→ can be calculated via birthday (so not permanently saved)

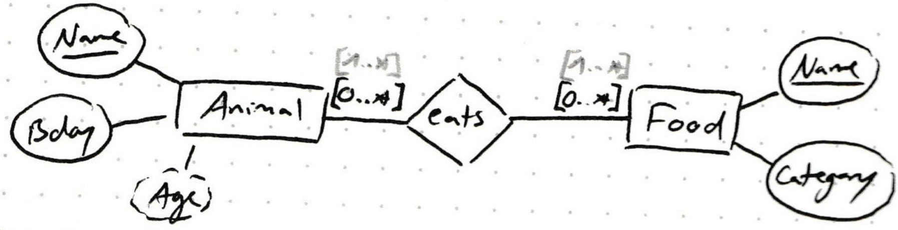
one-to-one



In this version: an animal can arrive without a Health Certificate, but every Health Cert is tied to an animal (or deleted on death)

→ other definition can also be correct (every animal has a HC if an animal dies, an HC can exist w/o it)

many-to-many



DB contd. 18.11.25

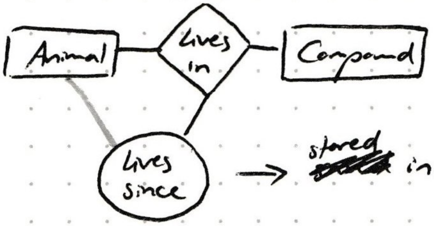
~~one-to-many~~ Attributes for one-to-many

Animal:

name	Compound	lives
X	A	2005
Y	A	2025
Z	B	1999

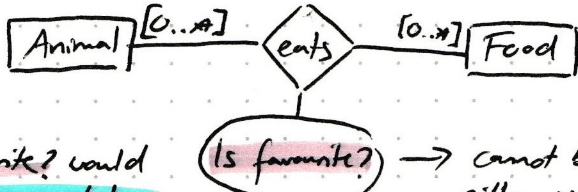
Compound:

name	...
A	
B	
C	
D	
E	



→ stored in Animal

for many-to-many



→ cannot be stored in either since mult foods is consumed by an animal and one food is consumed by mult animals

- Is favorite? would be its own relation

Eats:

Animal	Food	isFav
X	A	yes
X	B	no
Y	E	no
Y	A	yes
X	D	yes

ϟ two fav foods?
→ no solution so far (via diagram or key combo)

Animal:

Name	Fav
X	E
Y	E
Z	B

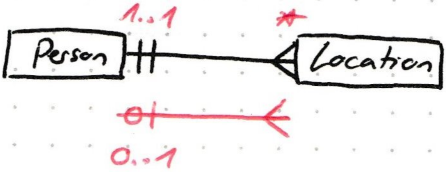
→ not completely wrong but should strive for the above

DB contd. 18/11/25

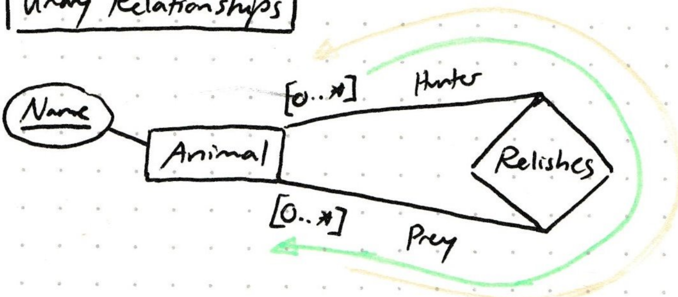
Crow's Foot Notation

– use it here to express logical design

→ has no many to many

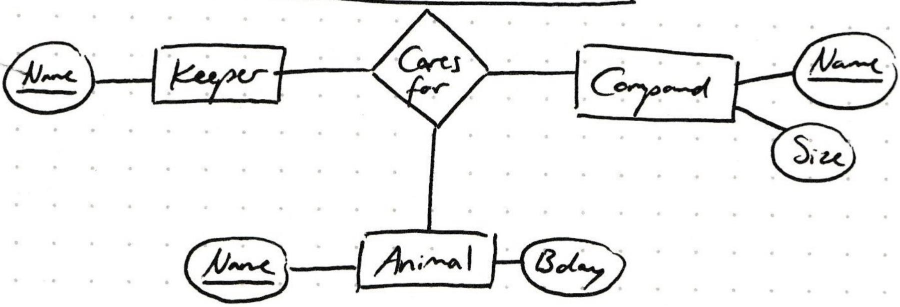


Unary Relationships

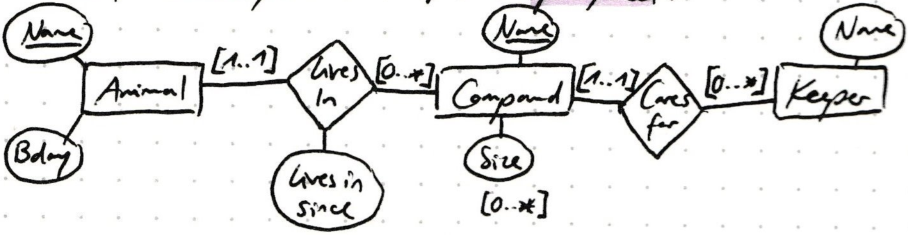


Animal is hunter of another Animal
Animal is prey of another Animal

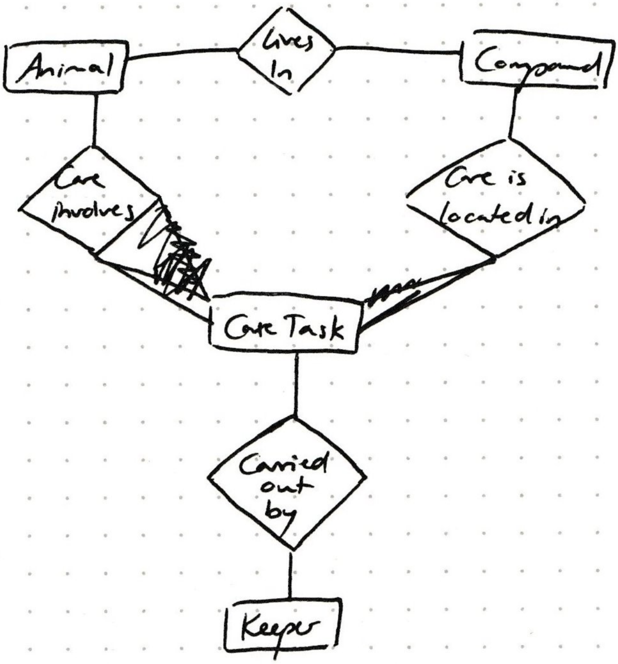
Relationship with more than two entities



→ quantizations are omitted as their meaning would be unclear
→ unary and binary relationships are prefered:



Alternative: (attributes omitted)



Specialization – Is-a-relationship

→ cf.